Introduction to Generative AI

Generative AI is a branch of artificial intelligence that creates new content such as text, images, audio, video, and code. Traditional AI systems mostly classify or predict, for example deciding whether an email is spam. Generative models instead produce new output based on patterns learned from very large amounts of training data.

Large language models, or LLMs, are the most widely used form of generative AI for text. They are built on the transformer architecture, which uses an attention mechanism to decide which earlier words matter most when predicting the next one.

During training, an LLM reads huge collections of text and learns to predict the next token, which is a small piece of a word. Repeating this prediction many times lets the model write paragraphs, answer questions, and generate code.

Training happens in stages. Pretraining teaches language and general knowledge. Fine-tuning and human feedback then teach the model to follow instructions and avoid harmful output. The context window is the amount of text a model can consider at once.

Embeddings and Vector Databases

An embedding is a list of numbers that captures the meaning of a piece of text. Texts with similar meaning have embeddings that are close together, so a computer can compare meaning using math instead of matching keywords.

For example, the sentences 'How do I reset my password?' and 'I forgot my login' have similar embeddings, while 'Best pizza in town' has a very different one. A common way to measure closeness is cosine similarity, where a higher score means a closer meaning.

A vector database stores embeddings and quickly finds the ones most similar to a query. Popular options include Chroma, FAISS, Qdrant, Pinecone, and pgvector. Documents and questions must be embedded with the same embedding model.

Embedding models only produce numbers. They do not write text, so they cannot hallucinate facts. They are used for semantic search, recommendations, clustering, and retrieval in RAG systems.

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, known as RAG, gives a language model the right information at question time. Instead of relying on memory, the model answers from text retrieved from your own documents.

RAG has two phases. In the indexing phase, documents are loaded, split into chunks, converted into embeddings, and stored in a vector database. In the answering phase, the question is embedded, the most similar chunks are retrieved, and they are added to the prompt as context.

A good RAG prompt tells the model to answer only from the provided context and to say it does not know when the answer is missing. This reduces hallucination, which is when a model confidently produces false information.

Retrieval quality decides answer quality. If the wrong chunks are retrieved, even a strong model cannot answer well. Chunk size, overlap, and the number of retrieved chunks are important settings to tune.

Common limitations include scanned PDFs that contain no readable text, tables that extract poorly, and documents that are too large to process without splitting. Always inspect the extracted text before building on it.